



Google Play App Analysis

EDA And Feature Engineering of Google Play Store Dataset

1) Problem statement.

Today, 1.85 million different apps are available for users to download. Android users have even more from which to choose, with 2.56 million available through the Google Play Store. These apps have come to play a huge role in the way we live our lives today. Our Objective is to find the Most Popular Category, find the App with largest number of installs, the App with largest size etc.

2) Data Collection.

The data consists of 20 column and 10841 rows.

Step1: In this Task I had to import python libraries such as

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings

warnings.filterwarnings("ignore")

%matplotlib inline
```

1. **Pandas:** Pandas 's library is helpful to create data frames in large data sets.
2. **NumPy:** NumPy's helpful to mathematical and statistical and linear algebra operations.
3. **Matplotlib and Seaborn:** help full to visualize the data

Step2: I was created a data frame using pandas

```
df=pd.read_csv('https://raw.githubusercontent.com/krishnaik06/playstore-Dataset/main/googleplaystore.csv')
df.head()
```

These are the Top 5 records of the data frame

	App	Category	Rating	Reviews	Size	Installs	Type	Price	Content Rating	Genres	Last Updated	Current Ver	Android Ver
0	Photo Editor & Candy Camera & Grid & ScrapBook	ART_AND_DESIGN	4.1	159	19M	10,000+	Free	0	Everyone	Art & Design	January 7, 2018	1.0.0	4.0.3 and up
1	Coloring book moana	ART_AND_DESIGN	3.9	967	14M	500,000+	Free	0	Everyone	Art & Design;Pretend Play	January 15, 2018	2.0.0	4.0.3 and up
2	U Launcher Lite – FREE Live Cool Themes, Hide ...	ART_AND_DESIGN	4.7	87510	8.7M	5,000,000+	Free	0	Everyone	Art & Design	August 1, 2018	1.2.4	4.0.3 and up
3	Sketch - Draw & Paint	ART_AND_DESIGN	4.5	215644	25M	50,000,000+	Free	0	Teen	Art & Design	June 8, 2018	Varies with device	4.2 and up
4	Pixel Draw - Number Art Coloring Book	ART_AND_DESIGN	4.3	967	2.8M	100,000+	Free	0	Everyone	Art & Design,Creativity	June 20, 2018	1.1	4.4 and up

Step3: Checking the records total how many records entered in the data set.

```
df.shape
(10841, 13)
```

In this data frame has 10841 records and Columns.

Step4:

1. checking the data types into the date frames then only we can easy to understand the data
 2. check the statistical summarize of the data.
- **df.info ()** → Data types of each column, memory usage, and missing values.
 - **df.describe ()** → Statistical summary (mean, median, min, max, quartiles, etc.) for numeric columns.

Step5: find the missing values in columns

```
##Missing Values
df.isnull().sum()

App          0
Category     0
Rating      1474
Reviews      0
Size         0
Installs     0
Type         1
Price        0
Content Rating  1
Genres       0
Last Updated  0
Current Ver   8
Android Ver   3
dtype: int64
```

Data Cleaning:

In the data cleaning I had been checked the data column wise and clean the data entire dataset for making good Accuracy of the dataset.

First, I was checking Reviews column in the column has numeric values it has object data type so I want to change the data type into integers but It was fail because in the data set has some where alphabet letters are there that's the reason it came error.

```
df['Reviews'].unique()

array(['159', '967', '87510', ..., '603', '1195', '398307'], dtype=object)

df['Reviews'].astype(int)

-----
ValueError                                Traceback (most recent call last)
Cell In[9], line 1
----> 1 df['Reviews'].astype(int)
```

ValueError: invalid literal for int () with base 10: '3.0M'

Now I check the column total how many rows recorded in numeric values and character values.

```
df['Reviews'].str.isnumeric().sum()
✓ 0.0s
np.int64(10840)

df[~df['Reviews'].str.isnumeric()]

App      Category  Rating  Reviews  Size  Installs  Type  Price  Content Rating  Genres  Last Updated  Current Ver  Android Ver
10472  Life Made Wi-Fi Touchscreen Photo Frame  1.9    19.0    3.0M  1,000+  Free  0  Everyone  NaN  February 11, 2018  1.0.19  4.0 and up  NaN
```

‘~’ tide is help to find the non-numeric values.

Before cleaning the data, I was copying the separate data frame because If I make any mistakes don’t need to go first step.

```
df_copy=df.copy()

df_copy=df_copy.drop(df_copy.index[10472])

df_copy[~df_copy['Reviews'].str.isnumeric()]

App      Category  Rating  Reviews  Size  Installs  Type  Price  Content Rating  Genres  Last Updated  Current Ver  Android Ver

## Convert Review Datatype to int
df_copy['Reviews']=df_copy['Reviews'].astype(int)
```

Finally, I converted data type of Review Column.

Second, I had checked the size column

```
df_copy['Size'].unique()
```

```
array(['19M', '14M', '8.7M', '25M', '2.8M', '5.6M', '29M', '33M', '3.1M',  
      '28M', '12M', '20M', '21M', '37M', '2.7M', '5.5M', '17M', '39M',  
      '31M', '4.2M', '7.0M', '23M', '6.0M', '6.1M', '4.6M', '9.2M',  
      '5.2M', '11M', '24M', 'Varies with device', '9.4M', '15M', '10M',  
      '1.2M', '26M', '8.0M', '7.9M', '56M', '57M', '35M', '54M', '201k',  
      '3.6M', '5.7M', '8.6M', '2.4M', '27M', '2.5M', '16M', '3.4M',  
      '8.9M', '3.9M', '2.9M', '38M', '32M', '5.4M', '18M', '1.1M',  
      '2.2M', '4.5M', '9.8M', '52M', '9.0M', '6.7M', '30M', '2.6M',  
      '7.1M', '3.7M', '22M', '7.4M', '6.4M', '3.2M', '8.2M', '9.9M',  
      '4.9M', '9.5M', '5.0M', '5.9M', '13M', '73M', '6.8M', '3.5M',  
      '4.0M', '2.3M', '7.2M', '2.1M', '42M', '7.3M', '9.1M', '55M',  
      '23k', '6.5M', '1.5M', '7.5M', '51M', '41M', '48M', '8.5M', '46M',  
      '8.3M', '4.3M', '4.7M', '3.3M', '40M', '7.8M', '8.8M', '6.6M',  
      '5.1M', '61M', '66M', '79k', '8.4M', '118k', '44M', '695k', '1.6M',  
      '6.2M', '18k', '53M', '1.4M', '3.0M', '5.8M', '3.8M', '9.6M',  
      '45M', '63M', '49M', '77M', '4.4M', '4.8M', '70M', '6.9M', '9.3M',  
      '10.0M', '8.1M', '36M', '84M', '97M', '2.0M', '1.9M', '1.8M',  
      '5.3M', '47M', '556k', '526k', '76M', '7.6M', '59M', '9.7M', '78M',  
      '72M', '43M', '7.7M', '6.3M', '334k', '34M', '93M', '65M', '79M',  
      '100M', '58M', '50M', '68M', '64M', '67M', '60M', '94M', '232k',  
      '99M', '624k', '95M', '8.5k', '41k', '292k', '11k', '80M', '1.7M',  
      '74M', '62M', '69M', '75M', '98M', '85M', '82M', '96M', '87M',  
      '71M', '86M', '91M', '81M', '92M', '83M', '88M', '704k', '862k',  
      '899k', '378k', '266k', '375k', '1.3M', '975k', '980k', '4.1M',  
      '89M', '696k', '544k', '525k', '920k', '779k', '853k', '720k',
```

In the column has millions and thousands and text values are there so I was replacing the data all in one type of values and change data type into float.

```
df_copy['Size'].isnull().sum()
```

```
0
```

```
df_copy['Size']=df_copy['Size'].str.replace('M','000')  
df_copy['Size']=df_copy['Size'].str.replace('k','')  
df_copy['Size']=df_copy['Size'].replace('Varies with device',np.nan)  
df_copy['Size']=df_copy['Size'].astype(float)
```

Third, I was checking the data set two columns that as **Installs** and **Price**

```
df_copy['Installs'].unique()
```

```
array(['10,000+', '500,000+', '5,000,000+', '50,000,000+', '100,000+',  
      '50,000+', '1,000,000+', '10,000,000+', '5,000+', '100,000,000+',  
      '1,000,000,000+', '1,000+', '500,000,000+', '50+', '100+', '500+',  
      '10+', '1+', '5+', '0+', '0'], dtype=object)
```

```
df_copy['Price'].unique()
```

```
array(['0', '$4.99', '$3.99', '$6.99', '$1.49', '$2.99', '$7.99', '$5.99',  
      '$3.49', '$1.99', '$9.99', '$7.49', '$0.99', '$9.00', '$5.49',  
      '$10.00', '$24.99', '$11.99', '$79.99', '$16.99', '$14.99',  
      '$1.00', '$29.99', '$12.99', '$2.49', '$10.99', '$1.50', '$19.99',  
      '$15.99', '$33.99', '$74.99', '$39.99', '$3.95', '$4.49', '$1.70',  
      '$8.99', '$2.00', '$3.88', '$25.99', '$399.99', '$17.99',  
      '$400.00', '$3.02', '$1.76', '$4.84', '$4.77', '$1.61', '$2.50',  
      '$1.59', '$6.49', '$1.29', '$5.00', '$13.99', '$299.99', '$379.99',  
      '$37.99', '$18.99', '$389.99', '$19.90', '$8.49', '$1.75',  
      '$14.00', '$4.85', '$46.99', '$109.99', '$154.99', '$3.08',  
      '$2.59', '$4.80', '$1.96', '$19.40', '$3.90', '$4.59', '$15.46',  
      '$3.04', '$4.29', '$2.60', '$3.28', '$4.60', '$28.99', '$2.95',  
      '$2.90', '$1.97', '$200.00', '$89.99', '$2.56', '$30.99', '$3.61',  
      '$394.99', '$1.26', '$1.20', '$1.04'], dtype=object)
```

These two columns I want to replace the data without special characters to these two columns in one time.

```
chars_to_remove=['+', ',', '$']  
cols_to_clean=['Installs', 'Price']  
for item in chars_to_remove:  
    for cols in cols_to_clean:  
        df_copy[cols]=df_copy[cols].str.replace(item, '')
```

After replacing the values, I had changed the data types **installs** column as convert to int and **price** as float

```
df_copy['Installs']=df_copy['Installs'].astype('int')  
df_copy['Price']=df_copy['Price'].astype('float')
```

Last, I was checking the data Last Update Column this column has calendar values in object types so I want to add three new columns month, year, date these three columns help to easy to analyze the data

```

## Handling Last update feature
df_copy['Last Updated'].unique()

array(['January 7, 2018', 'January 15, 2018', 'August 1, 2018', ...,
      'January 20, 2014', 'February 16, 2014', 'March 23, 2014'],
      dtype=object)

df_copy['Last Updated']=pd.to_datetime(df_copy['Last Updated'])
df_copy['Day']=df_copy['Last Updated'].dt.day
df_copy['Month']=df_copy['Last Updated'].dt.month
df_copy['Year']=df_copy['Last Updated'].dt.year

```

Now my cleaning steps had completed in the data and check the data types

```

df_copy.info()

<class 'pandas.core.frame.DataFrame'>
Int64Index: 10840 entries, 0 to 10840
Data columns (total 16 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   App                   10840 non-null  object
 1   Category              10840 non-null  object
 2   Rating                9366 non-null   float64
 3   Reviews               10840 non-null  int64
 4   Size                  9145 non-null   float64
 5   Installs              10840 non-null  int64
 6   Type                  10839 non-null  object
 7   Price                 10840 non-null  float64
 8   Content Rating        10840 non-null  object
 9   Genres                10840 non-null  object
10  Last Updated          10840 non-null  datetime64[ns]
11  Current Ver           10832 non-null  object
12  Android Ver           10838 non-null  object
13  Day                   10840 non-null  int64
14  Month                 10840 non-null  int64
15  Year                  10840 non-null  int64
dtypes: datetime64[ns](1), float64(3), int64(5), object(7)
memory usage: 1.4+ MB

```

Now I seen the data before cleaning and After cleaning:

Before cleaning:

	App	Category	Rating	Reviews	Size	Installs	Type	Price	Content Rating	Genres	Last Updated	Current Ver	Android Ver
0	Photo Editor & Candy Camera & Grid & ScrapBook	ART_AND_DESIGN	4.1	159	19M	10,000+	Free	0	Everyone	Art & Design	January 7, 2018	1.0.0	4.0.3 and up
1	Coloring book moana	ART_AND_DESIGN	3.9	967	14M	500,000+	Free	0	Everyone	Art & Design;Pretend Play	January 15, 2018	2.0.0	4.0.3 and up
2	U Launcher Lite – FREE Live Cool Themes, Hide ...	ART_AND_DESIGN	4.7	87510	8.7M	5,000,000+	Free	0	Everyone	Art & Design	August 1, 2018	1.2.4	4.0.3 and up
3	Sketch - Draw & Paint	ART_AND_DESIGN	4.5	215644	25M	50,000,000+	Free	0	Teen	Art & Design	June 8, 2018	Varies with device	4.2 and up
4	Pixel Draw - Number Art Coloring Book	ART_AND_DESIGN	4.3	967	2.8M	100,000+	Free	0	Everyone	Art & Design;Creativity	June 20, 2018	1.1	4.4 and up

After Cleaning the Data:

```
df_copy.head()
```

	App	Category	Rating	Reviews	Size	Installs	Type	Price	Content Rating	Genres	Last Updated	Current Ver	Android Ver	Month	Day	Year
0	Photo Editor & Candy Camera & Grid & ScrapBook	ART_AND_DESIGN	4.1	159	19000.0	10000	Free	0.0	Everyone	Art & Design	2018-01-07	1.0.0	4.0.3 and up	1	7	2018
1	Coloring book moana	ART_AND_DESIGN	3.9	967	14000.0	500000	Free	0.0	Everyone	Art & Design;Pretend Play	2018-01-15	2.0.0	4.0.3 and up	1	15	2018
2	U Launcher Lite – FREE Live Cool Themes, Hide ...	ART_AND_DESIGN	4.7	87510	8.7	5000000	Free	0.0	Everyone	Art & Design	2018-08-01	1.2.4	4.0.3 and up	8	1	2018
3	Sketch - Draw & Paint	ART_AND_DESIGN	4.5	215644	25000.0	50000000	Free	0.0	Teen	Art & Design	2018-06-08	Varies with device	4.2 and up	6	8	2018
4	Pixel Draw - Number Art Coloring Book	ART_AND_DESIGN	4.3	967	2.8	100000	Free	0.0	Everyone	Art & Design,Creativity	2018-06-20	1.1	4.4 and up	6	20	2018

Cleaning data has saved for future purpose to analyze the reports.

```
df_copy.to_csv("Google_Clean_Data.csv")
```

EXPLORATORY DATA ANALYSIS

After Cleaning the data first, I had to delete duplicates in the data.

```
df_copy[df_copy.duplicated(['App'])]
```

→ return duplicates

```
df_copy=df_copy.drop_duplicates(subset=['App'],keep='first')
```

Subset is identifying the columns, keep is store the first records and drop duplicate function delete the second records and keep the first records

Feature information of the data set categorical and numeric features to analysis.

```
numeric_Features=[feature for feature in df_copy.columns if df_copy[feature].dtype !='object']
Categoric_feature=[feature for feature in df_copy.columns if df_copy[feature].dtype == 'object']
#print Columns:
print('we have {} numerical features: {}'.format(len(numeric_Features),numeric_Features))
print('\nwe have {} Categoricall features: {}'.format(len(Categoric_feature),Categoric_feature))
```

```
we have 9 numerical features: ['Rating', 'Reviews', 'Size', 'Installs', 'Price', 'Last Updated', 'Month', 'Day', 'Year']
```

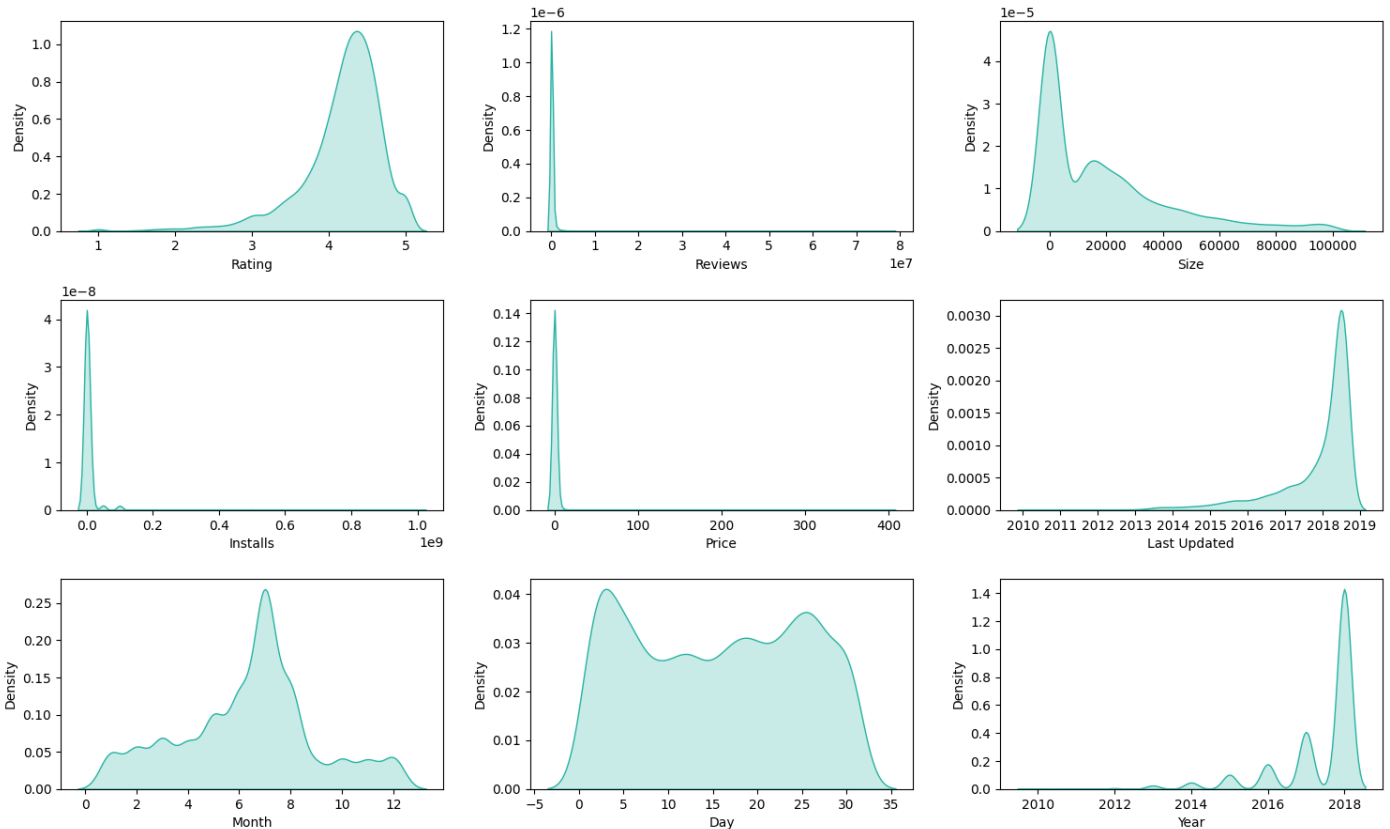
```
we have 7 Categoricall features: ['App', 'Category', 'Type', 'Content Rating', 'Genres', 'Current Ver', 'Android Ver']
```

Now Analysis of the feature numeric columns data.

```
## proportion Of count Data on Numerical Columns
plt.figure(figsize=(15,15))
plt.suptitle('Univariate Analysis of Numerical Features',fontsize=20, fontweight='bold',alpha=0.8,y=1.)

for i in range(0, len(numeric_Features)):
    plt.subplot(5,3, i+1)
    sns.kdeplot(x=df_copy[numeric_Features[i]], shade=True, color="#26b3a3")
    plt.xlabel(numeric_Features[i])
    plt.tight_layout()
```

Univariate Analysis of Numerical Features



Observations

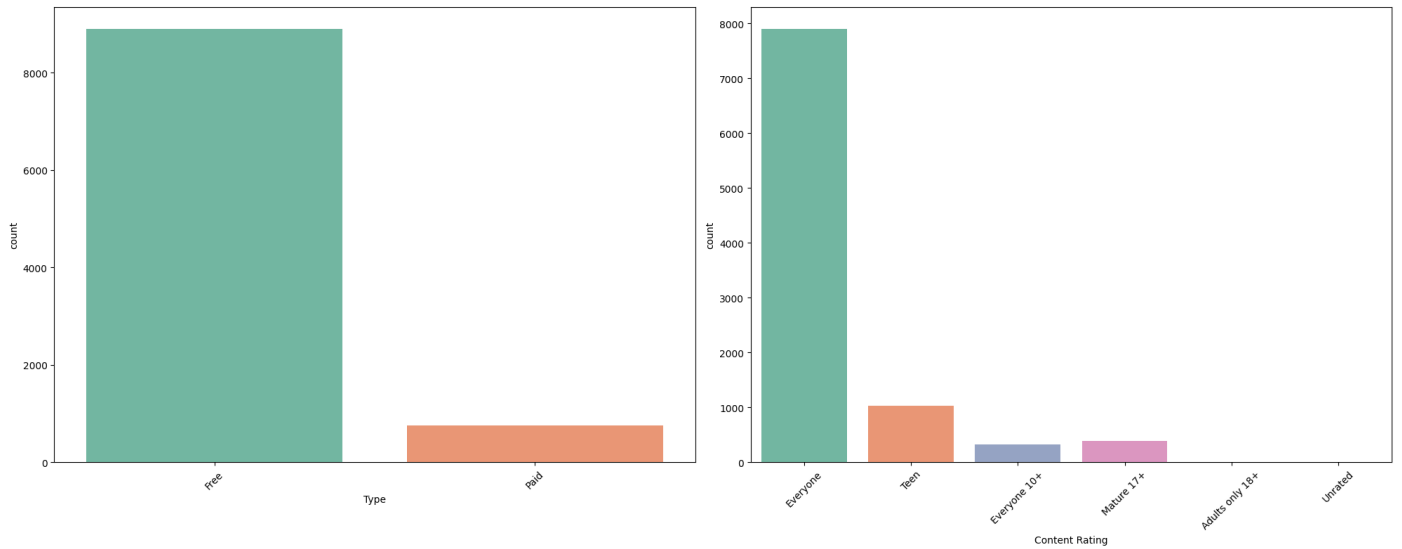
- Rating, Last Updated and Year is Left Skewed while reviews, Size, Installs and Price are Right Skewed

Now Analysis of the feature Categorical columns data.

```
# categorical columns
plt.figure(figsize=(20, 15))
plt.suptitle('Univariate Analysis of Categorical Features', fontsize=20, fontweight='bold', alpha=0.8, y=1.)
category = [ 'Type', 'Content Rating']## column names
for i in range(0, len(category)):
    plt.subplot(2, 2, i+1)
    sns.countplot(x=df_copy[category[i]],palette="Set2")

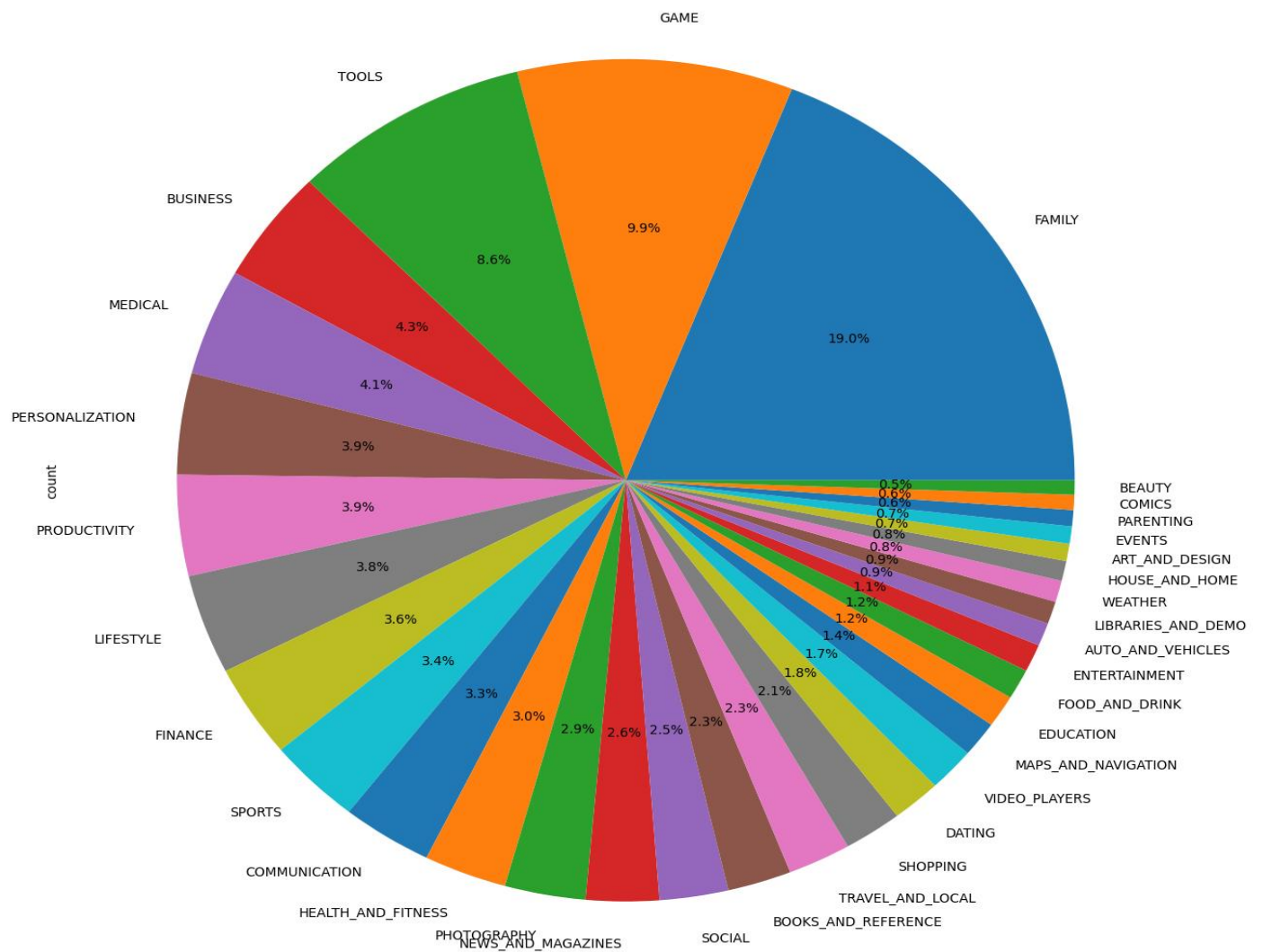
    plt.xticks(rotation=45)
    plt.tight_layout()
```


Univariate Analysis of Categorical Features



Find the large and least category values count

```
df_copy['Category'].value_counts().plot.pie(y=df_copy['Category'],figsize=(15,16),autopct='%1.1f%%')
```



Observations

1. There are more kinds of apps in play store which are under category of family, games & tools
2. Beauty, comics, arts and weather kinds of apps are very less in play store

top 10 app Categories

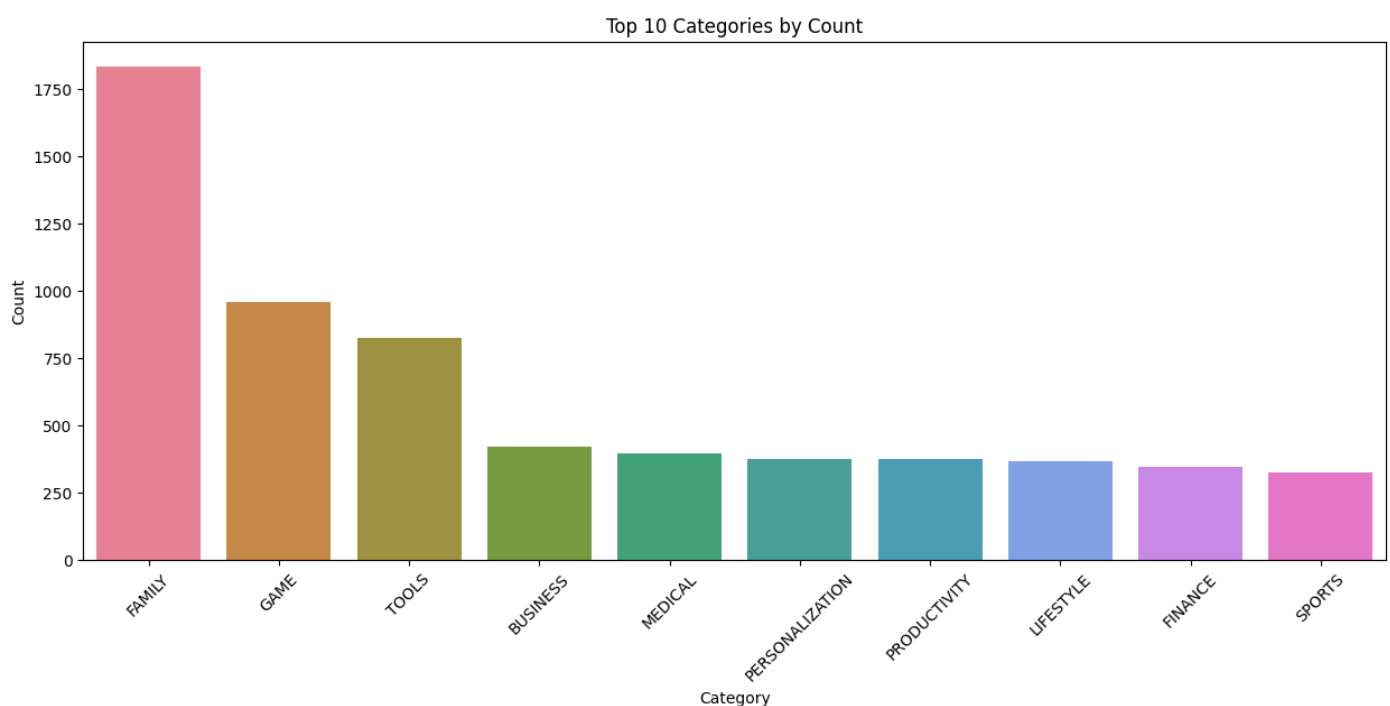
```
# top 10 app Categories
# Get category counts as a DataFrame
category = df_copy['Category'].value_counts().reset_index()

# Rename columns properly
category.columns = ['Category', 'Count']

# Take top 10
top10 = category.head(10)
```

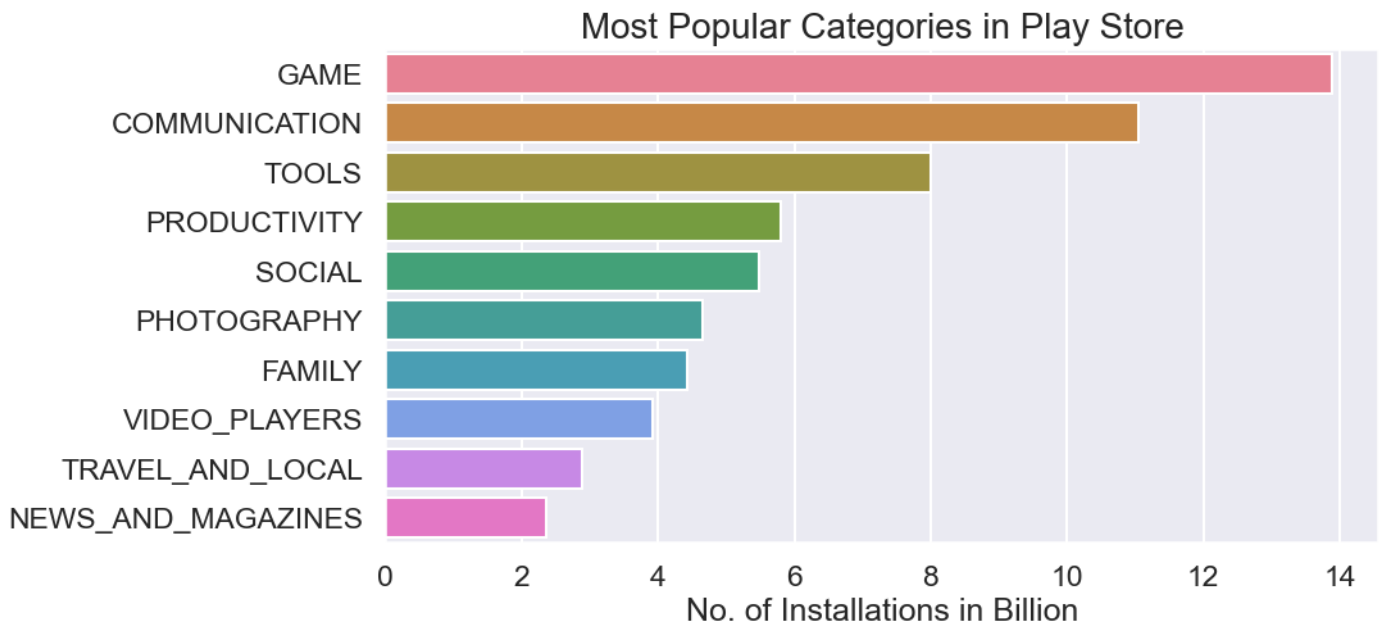
```
import seaborn as sns
import matplotlib.pyplot as plt
plt.figure(figsize=(15,6))
sns.barplot(
    x='Category',
    y='Count',
    data=top10,
    palette='husl'
)

plt.xticks(rotation=45) # rotate labels for readability
plt.title("Top 10 Categories by Count")
plt.show()
```



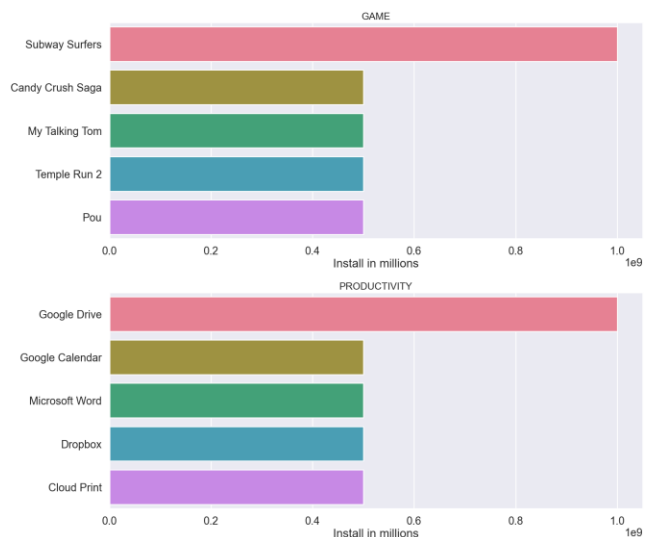
3.Which Category has largest number of installations?

```
df_cat_inst= df_copy.groupby(['Category'])['Installs'].sum().sort_values(ascending=False).reset_index()
df_cat_inst.Installs= df_cat_inst.Installs/1000000000
dfhi=df_cat_inst.head(10)
plt.figure(figsize=(10,5))
sns.set_context('talk')
sns.set_style('darkgrid')
ax= sns.barplot(x='Installs',y='Category',data=dfhi,palette='husl')
ax.set_xlabel("No. of Installations in Billion")
ax.set_ylabel('')
ax.set_title("Most Popular Categories in Play Store", size = 20)
```



4. what are the top 5 most installed app in GAME, 'COMMUNICATION', 'PRODUCTIVITY', 'SOCIAL' these categories?

```
dfa= df_copy.groupby(['App','Category'])['Installs'].sum().sort_values(ascending=False).reset_index()
dfa.rename(columns={'Installs':'Install in millions'},inplace=True)
apps=['GAME', 'COMMUNICATION', 'PRODUCTIVITY', 'SOCIAL']
sns.set_context('poster')
sns.set_style('darkgrid')
plt.figure(figsize=(40,30))
for i, app in enumerate(apps):
    df2=dfa[dfa.Category == app]
    df3=df2.head(5)
    plt.subplot(4,2, i+1)
    sns.barplot(data=df3,x='Install in millions',y='App',palette='husl')
    plt.ylabel('')
    plt.title(app,size=20)
plt.tight_layout()
plt.subplots_adjust()
plt.show()
```



5.how many apps are there in Google play store which get 5 ratings

```
rating = df_copy.groupby(['Category','Installs', 'App'])['Rating'].sum().sort_values(ascending = False).reset_index()
print(f"apps getting in 5 star raating: {rating[rating.Rating == 5.0].shape[0]}")
print('\n')
print("Highest Category of 5 Star rating: ")
rating.head(1)
```

apps getting in 5 star raating: 271

Highest Category of 5 Star rating:

	Category	Installs	App	Rating
0	FAMILY	10	DN Employee	5.0